

# SeaBIOS & BIOS Internals

A Comprehensive Guide for Operating System Developers

- BIOS Architecture
- x86 Real Mode
- OS Development

## 1. What is BIOS / SeaBIOS?

The **BIOS** (Basic Input/Output System) is the *first code that executes* when an x86 computer powers on. It initializes hardware, performs diagnostics (POST), and hands control to the bootloader. **SeaBIOS** is an open-source implementation of a legacy BIOS, widely used in QEMU, coreboot, and virtualized environments.

### Why study BIOS for OS development?

Understanding BIOS is crucial because your OS kernel receives the machine in the state the BIOS left it. You need to know: what memory is available, what interrupt vectors are set up, how the A20 gate works, and what hardware is already initialized.

### Key Responsibilities of BIOS

| PHASE | DESCRIPTION | SEABIOS SOURCE |
|-------|-------------|----------------|
|-------|-------------|----------------|

|                              |                                            |                             |
|------------------------------|--------------------------------------------|-----------------------------|
| Power-On Self Test (POST)    | Detect and initialize CPU, memory, chipset | src/post.c , src/memmap.c   |
| Hardware Enumeration         | Scan PCI bus, detect storage, USB, display | src/pciinit.c , src/ata.c   |
| Interrupt Vector Table (IVT) | Set up real-mode interrupt handlers        | src/ivt.c , src/romlayout.S |
| Option ROM Execution         | Run ROMs from PCI cards (VGA, NIC, RAID)   | src/optionroms.c            |
| Boot Device Selection        | Find bootable device, load MBR/bootloader  | src/boot.c                  |

## 2. CPU State When BIOS Starts

When the CPU comes out of reset, it's in a very specific state. Understanding this is fundamental:

| REGISTER | INITIAL VALUE | MEANING                                 |
|----------|---------------|-----------------------------------------|
| CS       | 0xF000        | Code Segment base                       |
| IP       | 0xFFFF        | Instruction Pointer                     |
| CS:IP    | 0xFFFF0       | Physical address = CS*16 + IP = 0xFFFF0 |
| CR0      | 0x60000010    | Real mode, no paging, FPU enabled       |

|           |                      |                                         |
|-----------|----------------------|-----------------------------------------|
| EFLAGS    | 0x00000002           | All flags cleared except reserved bit 1 |
| GDTR/IDTR | Base=0, Limit=0xFFFF | GDT/IDT not set up                      |

### SeaBIOS Reset Vector Code

```
/* From SeaBIOS: src/romlayout.S - Entry point at 0xFFFF0 */

// The CPU jumps to physical address 0xFFFF0 (0xF000:0xFFFF0)
// There's only 16 bytes left in the address space, so we immediately jump

        .org 0xffff          // Place code at offset 0xFFFF in ROM
        .globl reset_vector
reset_vector:
        ljmpw    $0xf000, $0xe05b // Far jump to actual POST entry

// At 0xFFFF0: ljmp $0xf000, $0xe05b (5 bytes: EA 5B E0 00 F0)
// This jumps to physical address 0xFE05B where the real POST code lives
```

 **Critical Detail:** The reset vector is at 0xFFFF0 — just 16 bytes from the top of the 1MB address space. The first instruction MUST be a far jump to reach the actual BIOS code located lower in the ROM.

## 3. The POST (Power-On Self Test) Flow

SeaBIOS's main initialization flow in `src/post.c` proceeds through well-defined phases. Here's the complete initialization sequence:

POWER ON / RESET

```
|
| → Phase 0: Early Setup [src/post.c: handle_post()]
|   | — Disable NMI (Non-Maskable Interrupt)
|   | — Clear EFLAGS (CLI - disable interrupts)
|   | — Set up temporary stack at known location
|   | — Clear BSS (zero-initialized data)
|   | — Initialize debug serial port (if configured)
|   | — Call platform-specific init (qemu_preinit / coreboot_preinit)
|
| → Phase 1: Chipset & Memory
|   | — enable_vx(): Initialize chipset bridges
|   | — smm_init(): System Management Mode setup
|   | — Detect CPU type/features (CPUID)
|   | — Setup MTRRs (Memory Type Range Registers)
|   | — ram_probe(): Detect available RAM size
|   | — malloc_init(): Set up BIOS heap
|   | — Setup EBDA (Extended BIOS Data Area)
|
| → Phase 2: IVT & BDA Setup
|   | — ivt_init(): Initialize Interrupt Vector Table at 0x00000
|   | — bda_init(): Initialize BIOS Data Area at 0x00400
|   | — Set up real-mode IVT entries (INT 0x10, 0x13, 0x15, 0x16, 0x1A...)
|   | — Set up BIOS32 Service Directory
|
| → Phase 3: Hardware Init
|   | — pci_init(): Enumerate PCI bus
|   | — vga_setup(): Initialize display (VGA/VESA)
|   | — ata_init(): Initialize ATA/IDE controllers
```

- | |— ahci\_init(): Initialize AHCI (SATA) controllers
- | |— usb\_init(): Initialize USB controllers & hubs
- | |— ps2\_init(): Initialize keyboard/mouse
- | |— serial\_init(): Initialize COM ports
- | |— floppy\_init(): Initialize floppy drives

#### |→ Phase 4: Option ROMs

- | |— Scan PCI devices for Option ROMs
- | |— Copy ROMs to RAM (shadowing)
- | |— Execute ROM in real-mode emulation
- | |— Examples: VGA BIOS, PXE ROM, RAID BIOS

#### |→ Phase 5: Boot Device Selection (BBS)

- | |— Enumerate bootable devices (bootprio\_find\_\*)
- | |— Apply boot order (user config / CMOS)
- | |— For each candidate boot device:
  - | | |— Read sector 0 (MBR) to 0x7C00
  - | | |— Verify 0xAA55 magic signature
  - | | |— If valid: JMP to 0x0000:0x7C00
- | |— If all fail: Display "No bootable device"

#### |→ Bootloader / OS takes over

🔑 Key Insight for OS Developers: When your bootloader or kernel starts at 0x7C00, the BIOS has already

## 4. Standard PC Memory Map (What BIOS Leaves You)

After BIOS initialization, the lower 1MB memory space is organized as follows. This is **critical knowledge** for kernel development:

**0x00000 - 0x003FF**

Real Mode IVT  
(1024 bytes, 256 entries × 4 bytes)

**0x00400 - 0x004FF**

BIOS Data Area (BDA)  
Keyboard flags, video mode, COM ports

**0x00500 - 0x07BFF**

Conventional Memory  
~30KB available, used by bootloader

**0x07C00 - 0x07DFF**

 Boot Sector Load Address  
512 bytes for MBR/VBR

**0x07E00 - 0x9FBFF**

Free Conventional Memory  
~608KB available for kernel use

**0x9FC00 - 0x9FFFF**

EBDA (Extended BIOS Data)  
1KB, variable size

**0xA0000 - 0xBFFFF**

 Video Memory  
128KB VGA framebuffer

**0xC0000 - 0xC7FFF**

VGA BIOS ROM  
32KB (typical)

**0xC8000 - 0xEFFFF**

Option ROMs / Adapter ROM  
160KB (NIC, RAID, SCSI)

**0xF0000 - 0xFFFFF**

System BIOS ROM  
64KB shadowed to RAM

## Reading the Memory Map via INT 0x15, EAX=0xE820

```

/* SeaBIOS implements INT 0x15, EAX=0xE820 to report memory map.
   This is the preferred way for bootloaders to get memory layout. */

// Input:
//   EAX = 0xE820
//   EDX = 0x534D4150 ('SMAP')
//   EBX = continuation value (0 for first call)
//   ECX = size of buffer (>= 20 for ACPI 3.0, >= 24 for ACPI 4.0)
//   ES:DI = pointer to buffer
//
// Output:
//   EAX = 0x534D4150 ('SMAP')
//   EBX = continuation value (0 means done)
//   ECX = bytes actually written
//   CF = 0 on success, 1 on error

struct e820_entry {
    uint64_t base_addr;    // Base address of memory region
    uint64_t length;      // Length in bytes
    uint32_t type;         // 1=Usable RAM, 2=Reserved, 3=ACPI, 4=ACPI NVS, 5=Bad
    uint32_t acpi_attrs;   // ACPI 3.0 extended attributes (if ECX >= 24)
};

// Example output (QEMU with 4GB RAM):
// [0] base=0x0000000000000000 Length=0x000000000009FC00 type=1 (Usable)
// [1] base=0x0000000000009FC00 Length=0x000000000000400 type=2 (Reserved/ACPI)
// [2] base=0x00000000000F0000 Length=0x000000000010000 type=2 (System ROM shadow)
// [3] base=0x000000000100000 Length=0x00000000BFEE0000 type=1 (Usable, ~3GB)

```

```
// [4] base=0x00000000BFFE0000 Length=0x000000000020000 type=2 (Possible MMIO)
// [5] base=0x0000000100000000 Length=0x0000000140000000 type=1 (Usable, 5GB above 4G)
```

## 5. The A20 Gate — The Famous Wrinkle

The A20 gate is a historical artifact from the 8086/80286 era that still affects modern systems. Understanding it is essential for OS development.

### Why it exists:

- The 8086 had 20 address lines (A0-A19), addressing 1MB (0x00000-0xFFFFF)
- Addresses above 1MB "wrapped around" due to the 20-bit limitation
- Some DOS programs relied on this wrap-around behavior
- The 80286 added A20-A23, enabling access up to 16MB
- For backward compatibility, the A20 line was gated — default OFF

```
/* A20 Gate: Controls whether address line 20 is forced to 0.
   When A20 is DISABLED: addresses wrap (like 8086)
   When A20 is ENABLED: full addressing (required for protected mode) */

// Traditional methods to enable A20:

// Method 1: Keyboard Controller (SeaBIOS uses this as fallback)
enable_a20_kbc:
    call    empty_8042    // Wait until input buffer empty
```



```
mov    $0xD1, %al      // Command: write output port
out     %al, $0x64
call   empty_8042
mov     $0xDF, %al      // Value: bit 1 = A20 enable
out     %al, $0x60
ret

// Method 2: Fast A20 Gate (System Control Port A)
enable_a20_fast:
in      $0x92, %al
or      $2, %al         // Set bit 1 (A20 gate)
out     %al, $0x92
ret

// Method 3: BIOS INT 0x15, AH=0x24
// Input: AL = 0x00 (disable), 0x01 (enable), 0x02 (query status)
```

 **SeaBIOS A20 handling:** See `src/fw/ps2port.c` and `src/hw/ps2port.c`. SeaBIOS tries Fast A20 first, then falls back to keyboard controller. Modern chipsets can also enable A20 through the northbridge.

## 6. BIOS Interrupt Services Available to Your Bootloader

---

Before switching to protected mode (which invalidates the real-mode IVT), your bootloader can use these BIOS services. SeaBIOS implements the standard set:

| INTERRUPT | SERVICE         | COMMON FUNCTIONS                                     | SEABIOS SOURCE          |
|-----------|-----------------|------------------------------------------------------|-------------------------|
| INT 0x10  | Video Services  | Set mode, write character, scroll, get cursor        | vgasrc/vgaentry.S       |
| INT 0x13  | Disk I/O        | Read/write sectors, get drive params, EDD extensions | src/disk.c , src/ata.c  |
| INT 0x14  | Serial Port     | Init, send, receive, status                          | src/serial.c            |
| INT 0x15  | System Services | Memory map (E820), A20 control, APM                  | src/system.c            |
| INT 0x16  | Keyboard        | Read keystroke, check status, get shift flags        | src/kbd.c               |
| INT 0x1A  | RTC / PCI       | Read/set clock, PCI BIOS, timer tick                 | src/clock.c , src/pci.c |

Example: Disk Read via INT 0x13

```
/* Read 1 sector from disk using INT 0x13 AH=0x02
   This is the classic method used by bootloaders to load the kernel */

read_sector:
    mov    $0x02, %ah      // AH = function 0x02 (read sectors)
    mov    $0x01, %al      // AL = number of sectors to read
    mov    $0x0000, %cx     // CH = cylinder (0), CL = sector (1)
    mov    $0x0000, %dx     // DH = head (0), DL = drive number (0x80=first HDD)
    mov    $0x1000, %bx     // ES:BX = destination buffer (0x1000:0x0000)
    int    $0x13

    // On return: CF=0 means success, CF=1 means error (AH=error code)
    jc     disk_error
```

```
// Extended Read (LBA) via INT 0x13 AH=0x42 - preferred method
extended_read:
    mov     $0x42, %ah      // Extended read function
    mov     $0x80, %dl      // Drive number
    mov     $disk_packet, %si // DS:SI → Disk Address Packet
    int     $0x13

// Disk Address Packet (DAP) structure:
disk_packet:
    .byte    0x10           // Packet size (16 bytes)
    .byte    0x00           // Reserved, must be 0
    .word    0x01           // Number of sectors to transfer
    .word    0x1000, 0x0000 // Offset:Segment of destination buffer
    .quad    0x00000001     // Starting LBA (8 bytes, little-endian)
```

## 7. Transitioning from BIOS to Your Kernel

This is the most critical moment in booting. The BIOS hands control to your code at `0x0000:0x7C00` in **real mode**. Here's what you must do:

Bootloader Flow (16-bit Real Mode → 32/64-bit Protected Mode)

0x7C00: Bootloader/Bootstrap starts

|

└─> Step 1: Relocate (if needed)

- | └─ Copy yourself above 0x7E00 if you need more space
- |
- | → **Step 2: Query Memory Map**
- | └─ Call INT 0x15, EAX=0xE820 to get available memory
- |
- | → **Step 3: Enable A20 Gate**
- | └─ Call BIOS INT 0x15 AH=0x24 or use keyboard controller
- |
- | → **Step 4: Load Kernel into Memory**
- | └─ Use INT 0x13 AH=0x42 (extended read) to load kernel sectors
- |
- | → **Step 5: Set Up Protected Mode Structures**
- | └─ Create GDT (Global Descriptor Table)
- | └─ Create page tables (if enabling paging)
- | └─ Set up initial stack
- |
- | → **Step 6: Enter Protected Mode**
- | └─ CLI (disable interrupts – IVT no longer valid)
- | └─ LGDT [gdtr] (load GDT register)
- | └─ Set CR0 bit 0 (PE = Protected Mode Enable)
- | └─ Far jump to flush prefetch queue, load CS with selector
- |
- | → **Step 7: 32-bit World**
- | └─ Reload segment registers (DS, ES, FS, GS, SS)
- | └─ Set up new stack in known-good memory
- | └─ Jump to kernel entry point
- |
- | → **Step 8: Optionally Enable Paging**
- | └─ Load CR3 with page directory address
- | └─ Set CR0 bit 31 (PG = Paging Enable)
- | └─ Now running with virtual memory!

## Minimal Protected Mode Transition Code

```

/* Minimal GDT for switching to 32-bit protected mode */

gdt_start:
    ; Null descriptor (required)
    .quad    0x0000000000000000

    ; Code segment descriptor (DPL=0, executable, 32-bit)
    .word    0xFFFF            ; Limit[0:15]
    .word    0x0000            ; Base[0:15]
    .byte    0x00              ; Base[16:23]
    .byte    0x9A              ; Access: P=1, DPL=00, S=1, Type=1010 (exec/read)
    .byte    0xCF              ; Flags: G=1, D=1 (32-bit), Limit[16:19]=1111
    .byte    0x00              ; Base[24:31]

    ; Data segment descriptor (DPL=0, writable, 32-bit)
    .word    0xFFFF            ; Limit[0:15]
    .word    0x0000            ; Base[0:15]
    .byte    0x00              ; Base[16:23]
    .byte    0x92              ; Access: P=1, DPL=00, S=1, Type=0010 (read/write)
    .byte    0xCF              ; Flags: G=1, D=1, Limit[16:19]=1111
    .byte    0x00              ; Base[24:31]
gdt_end:

gdtr:
    .word    gdt_end - gdt_start - 1    ; GDT limit
    .long    gdt_start                  ; GDT base address

/* Switch to protected mode */
switch_to_pm:
    cli                                // Disable interrupts
    lgdt     [gdtr]                    // Load GDT register

    mov     %cr0, %eax
    or      $0x01, %eax                // Set PE bit
    mov     %eax, %cr0

```

```
// Far jump to flush prefetch & load 32-bit CS
ljmp  $0x08, $protected_mode_entry // 0x08 = code segment selector

.code32
protected_mode_entry:
    // Set up data segments
    mov  $0x10, %ax           // 0x10 = data segment selector
    mov  %ax, %ds
    mov  %ax, %es
    mov  %ax, %fs
    mov  %ax, %gs
    mov  %ax, %ss

    // Set up stack
    mov  $0x90000, %esp

    // Jump to kernel – you're now in 32-bit protected mode!
    call kernel_main
```

### 💡 Pro Tips:

- Always call `CLI` before entering protected mode – BIOS interrupt handlers are real-mode code and will crash in protected mode.
- The far jump after setting `CR0` is required – it flushes the prefetch queue and loads CS with a real protected-mode selector.

- SeaBIOS sets up the EBDA size – you can read it from BDA offset 0x0E to know exactly where conventional memory ends.
- The 0xAA55 signature at offset 510 in the MBR is checked by SeaBIOS in src/boot.c: boot\_add\_bootable\_device() .

## 8. Standard Hardware Assumptions

For OS development targeting "standard hardware" (as QEMU presents it), here's what you can rely on:

| COMPONENT | STANDARD CONFIGURATION                                                        | I/O PORTS / MMIO      |
|-----------|-------------------------------------------------------------------------------|-----------------------|
| 8259 PIC  | Master at 0x20, Slave at 0xA0<br>IRQ0=Timer, IRQ1=Keyboard, IRQ14=Primary IDE | 0x20-0x21 , 0xA0-0xA1 |

|                         |                                                      |                                |
|-------------------------|------------------------------------------------------|--------------------------------|
| 8253/8254 PIT           | Channel 0: System timer (~18.2 Hz)                   | 0x40-0x43                      |
|                         | Channel 1: DRAM refresh                              |                                |
|                         | Channel 2: PC speaker                                |                                |
| 16550 UART              | COM1 at 0x3F8 (IRQ4)                                 | 0x3F8-0x3FF , 0x2F8-0x2FF      |
|                         | COM2 at 0x2F8 (IRQ3)                                 |                                |
| 8042 PS/2 Controller    | Keyboard + Mouse, also A20 gate                      | 0x60 (data), 0x64 (status/cmd) |
| VGA                     | Text mode: 0xB8000 (color)                           | 0x3C0-0x3DF , 0xB8000          |
|                         | CRTC registers, sequencer, graphics controller       |                                |
| PCI Configuration Space | Accessed via I/O ports 0xCF8 (addr) and 0xCFC (data) | 0xCF8 , 0xCFC                  |
| CMOS/RTC                | Real-time clock and configuration memory             | 0x70 (index), 0x71 (data)      |
| DMA Controller (8237)   | Two cascaded controllers (8 channels)                | 0x00-0x0F , 0xC0-0xDF          |
| ACPI                    | RSDP found via BIOS or memory scan ("RSD PTR ")      | EBDA or 0xE0000-0xFFFFF        |

## 9. Debugging BIOS & Early Boot Code

### Useful SeaBIOS Debug Techniques



- **Serial console:** SeaBIOS outputs debug info to COM1 at 115200 baud. Add `-debugcon file:seabios.log` in QEMU.
- **QEMU tracing:** `qemu -d cpu_reset,int,mmu,in_asm -D trace.log`
- **GDB with QEMU:** `qemu -s -S` then `gdb -ex "target remote localhost:1234"`
- **SeaBIOS debug levels:** Set in Kconfig or via `debug_level` variable (0-10)

```
/* SeaBIOS uses dprintf() for debug logging - src/output.c */

// Debug Levels:
// 0 - essential messages only
// 1 - warnings
// 3 - informative messages
// 5 - verbose hardware detection
// 8 - extremely verbose (all register dumps)

// In serial output, you'd see messages like:
// "Found 8192 MB of RAM"
// "ATA 0: Hitachi HDS721010CLA332 ATA-8 Hard-Disk (931 GiBytes)"
// "Scan for option roms"
// "Booting from Hard Disk... Booting from 0000:7c00"
```

## 10. Roadmap: From BIOS to Your First OS Kernel

Recommended Learning Path for OS Development:

### Phase 1: Understanding the Environment

- └─ Write a simple boot sector that prints "Hello" via BIOS INT 0x10
- └─ Use INT 0x13 to load additional sectors
- └─ Call INT 0x15 E820 to get the memory map
- └─ Display memory regions on screen

### Phase 2: Protected Mode Transition

- └─ Set up a minimal GDT
- └─ Enter 32-bit protected mode
- └─ Write directly to VGA text buffer at 0xB8000
- └─ Set up a basic IDT with exception handlers
- └─ Implement interrupt handlers for keyboard, timer

### Phase 3: Basic Kernel

- └─ Physical memory allocator (bitmap or free list)
- └─ Set up paging (identity map first 4MB)
- └─ Higher-half kernel (link above 0xC0000000)
- └─ Basic process/thread scheduler
- └─ System call interface

### Phase 4: Hardware Drivers

- └─ Program the 8259 PIC (remap IRQs)
- └─ Program the PIT/APIC timer
- └─ PS/2 keyboard driver with scancode handling
- └─ Serial port driver for logging
- └─ (Advanced) AHCI/SATA storage driver

 Common Pitfalls:

- **Forgetting CLI:** An interrupt in the middle of protected mode transition will crash hard.
- **Stack placement:** Don't place the stack where it overlaps your code or IVT.
- **A20 not enabled:** Odd-numbered megabytes will be inaccessible.
- **Segment limits:** In protected mode, you need valid GDT entries before touching data segments.
- **IDT not set up:** Any exception (including NMI) will triple-fault and reboot.

# 11. Key SeaBIOS Source Files to Study

| FILE                      | PURPOSE                                         | WHY IT'S IMPORTANT FOR OS DEV               |
|---------------------------|-------------------------------------------------|---------------------------------------------|
| src/post.c                | Main POST initialization sequence               | Shows exact order of hardware init          |
| src/boot.c                | Boot device selection & MBR loading             | Shows what state the machine is in at boot  |
| src/romlayout.S           | Reset vector, real-mode entry points, IVT setup | Understanding CPU init and IVT layout       |
| src/memmap.c              | Memory detection and E820 map construction      | Learn how memory map is built               |
| src/pciinit.c             | PCI bus enumeration and resource allocation     | PCI is essential for modern hardware access |
| src/ata.c /<br>src/ahci.c | Disk controller initialization and I/O          | Storage is critical for loading your kernel |
| src/fw/smp.c              | Multiprocessor initialization (AP startup)      | Understanding multi-core boot process       |
| vgasrc/                   | VGA BIOS implementation                         | Display init before your kernel takes over  |

src/fw/biostables.c

ACPI/SMBIOS/MPTable construction

Modern OSes need these tables for hardware enumeration

## 12. Summary: What Your Kernel Receives

When SeaBIOS (or any BIOS) jumps to your bootloader at 0x7C00 , here's a concise checklist of the machine state:

| ITEM             | STATE                                                                 |
|------------------|-----------------------------------------------------------------------|
| CPU Mode         | Real mode (16-bit), A20 enabled by BIOS                               |
| Interrupts       | Enabled (IF=1). IVT at 0x00000 with BIOS handlers                     |
| DL Register      | Boot drive number (0x00=floppy, 0x80=first HDD)                       |
| Stack            | BIOS stack near top of conventional memory (~0x7BF0)                  |
| CS:IP            | Usually 0x0000:0x7C00 (some BIOS use 0x07C0:0x0000)                   |
| Memory below 1MB | As shown in the memory map — IVT, BDA, your code at 0x7C00            |
| PCI Bus          | Enumerated and configured. Use INT 0x1A or direct config space access |

|                |                                                       |
|----------------|-------------------------------------------------------|
| <b>Display</b> | Text mode 80×25 at 0xB8000 (color) or 0xB0000 (mono)  |
| <b>VGA</b>     | Option ROM executed, VGA initialized in standard mode |
| <b>EBDA</b>    | Allocated, size at BDA offset 0x0E (in KB)            |

 **The Golden Rule of OS Development:** Trust nothing the BIOS gave you. Verify the memory map, remap the PIC, set up your own GDT/IDT, and switch to protected mode as soon as practical. The BIOS is a trampoline — use it to jump into your world, then build your own.

SeaBIOS is open source — clone it from [git://git.seabios.org/seabios.git](https://git.seabios.org/seabios.git) and explore.

Recommended reading: Intel Manual Vol. 3A (System Programming), "Writing a Simple Operating System from Scratch" by Nick Blundell, and the OSDev Wiki.

Generated for OS development learning purposes. SeaBIOS is GPLv3 licensed by Kevin O'Connor and contributors.